Module Introduction

ফিচার ইঞ্জিনিয়ারিং কী?

ফিচার ইঞ্জিনিয়ারিং হলো কাঁচা ডেটাকে এমন ফরম্যাটে রূপান্তর করা, যাতে মেশিন লার্নিং মডেল সহজেই তা বুঝতে পারে এবং শিখতে পারে।

উদাহরণ:

আপনার কাছে "নাম" আর "বয়স" আছে। মডেল "নাম" থেকে কিছু শিখবে না, কিন্তু বয়স থেকে শিখবে। তাই দরকারি ফিচার আলাদা করা, অনুপস্থিত মান পূরণ করা, ক্যাটাগরিকে নাম্বারে রূপান্তর করা ইত্যাদি।

ফিচার ইঞ্জিনিয়ারিংয়ের মূল ধাপগুলো:

১. ডেটা স্প্লিটিং **(Train-Test Split)**

- ডেটাকে দুই ভাগে ভাগ করা: ট্রেনিং সেট (৭০-৮০%) ও টেস্টিং সেট (২০-৩০%)
- ট্রেনিং সেটে মডেল শেখে, টেস্টিং সেটে মডেল যাচাই করা হয়
- ফিচার (X) ও টার্গেট (y) আলাদা করে তারপর স্প্লিট করতে হয়

২. মিসিং ভ্যালু হ্যান্ডলিং

- ডেটাসেটের ফাঁকা জায়গাগুলো পূরণ করা বা সরিয়ে ফেলা
- নিউমেরিক্যাল ডেটার জন্য: গড় (মিন), মাঝের মান (মিডিয়ান), বা সবচেয়ে বেশি আসা মান (মোড) বসানো
- ক্যাটেগরিক্যাল ডেটার জন্য: সবচেয়ে বেশি আসা ক্যাটাগরি বা "Unknown" লেবেল বসানো

৩. ফিচার এক্সট্রাকশন

- প্রয়োজনীয় ফিচারগুলো আলাদা করা
- অপ্রয়োজনীয় ফিচার বাদ দেওয়া

৪. আউটলায়ার হ্যান্ডলিং **(পরবর্তী পর্বে)**

- অস্বাভাবিক বা চরম মানগুলো শনাক্ত ও হ্যান্ডল করা

৫. স্কেলিং ও নরমালাইজেশন **(পরবর্তী পর্বে)**

- বিভিন্ন ইউনিটের ডেটাকে একই স্কেলে নিয়ে আসা

৬. এনকোডিং **(পরবর্তী পর্বে)**

- ক্যাটেগরিক্যাল ডেটাকে নম্বরে রূপান্তর করা

Feature Engineering and its steps

ফিচার ইঞ্জিনিয়ারিং কী এবং কেন করব?

ফিচার ইঞ্জিনিয়ারিং হলো সেই প্রক্রিয়া, যার মাধ্যমে কাঁচা (Raw) ডেটাকে মেশিন লার্নিং মডেল বোঝার উপযোগী করে তোলা হয়। মডেল শুধু সংখ্যা বোঝে, তাই যেকোনো ডেটা (টেক্সট, ছবি, ক্যাটাগরি) – সবকিছুকে সংখ্যায় রূপান্তর করতে হয়।

কেন এত জরুরি?

রিয়েল ওয়ার্ল্ডের ডেটা কখনো পারফেক্ট হয় না। সেখানে থাকে:

- অপ্রয়োজনীয় কলাম (যেমন: প্যাসেঞ্জার আইডি, নাম)
- ফাঁকা জায়গা (Missing Values)

টেক্সট আকারে ক্যাটাগরি (যেমন: শহরের নাম, লিঙ্গ)
ছোট-বড় অসংগত মান (আউটলায়ার)
বিভিন্ন স্কেলের সংখ্যা (যেমন: সিজিপিএ ১-৪, কিন্তু স্যালারি ২০,০০০-১,০০,০০০)

উদাহরণ (টাইটানিক ডেটাসেট):

PassengerId বা Name দেখে কেউ বাঁচবে কিনা – তার কোনো সম্পর্ক নেই। এগুলো ফিচার না।
Sex (male/female) – টেক্সট, মডেল বোঝে না। এটাকে ০/১ করতে হবে।
Age কলামে অনেক ফাঁকা মান – পূরণ না করলে মডেল error দেবে।

মূল কথা: ফিচার ইঞ্জিনিয়ারিং না করলে মডেল শিখতে পারে না, আর ভুল শিখলে প্রেডিকশন ভুল হয়।

ফিচার ইঞ্জিনিয়ারিংয়ের সম্পূর্ণ ধাপ

নিচের ধাপগুলো ফলো করলেই একটি পূর্ণাঙ্গ ফিচার ইঞ্জিনিয়ারিং প্রক্রিয়া পাবেন।

ধাপ ১: স্টার্টার – অপ্রয়োজনীয় কলাম বাদ দেওয়া (Feature Selection - Basic)

কাজ: যেসব কলাম টার্গেটের সাথে সম্পর্কিত না, সেগুলো ডিলিট করে ফেলা।

উদাহরণ: টাইটানিকে PassengerId, Name, Ticket Number – এগুলো সরিয়ে দিন।

কেন? এরা মডেলকে বিভ্রান্ত করে, শেখার কোনো যোগ্যতা নেই।

ধাপ ২: ফিচার (X) ও টার্গেট (y) আলাদা করা

কাজ: ডেটাসেট থেকে ইনপুট ফিচারগুলোকে `X` এবং আউটপুট/প্রেডিক্ট করার জিনিসটাকে `y` তে রাখা।

উদাহরণ: টাইটানিকে বাঁচবে কি না – সেটা `y` (Survived) বাকি সব কলাম `X`।

কেন? মডেল ট্রেনিং-এর সময় স্পষ্টভাবে ইনপুট-আউটপুট আলাদা করতে হয়।

ধাপ ৩: ট্রেন-টেস্ট স্প্লিট করা

কাজ: পুরো ডেটাকে ভাগ করা – ৭০-৮০% ট্রেনিং (মডেল শেখার জন্য), বাকি ২০-৩০% টেস্টিং (মডেল যাচাইয়ের জন্য)

কেন? ট্রেনিং ডেটাতে শিখিয়ে টেস্ট ডেটাতে পরীক্ষা না করলে জানবেন না মডেল আসলেই ভালো কাজ করছে কিনা (overfitting হবে)।

গুরুত্বপূর্ণ নিয়ম: প্রথমে স্প্লিট, তারপর বাকি সব স্টেপ (মিসিং ভ্যালু পূরণ, স্কেলিং) ট্রেনিং ডেটা থেকে শিখে টেস্ট ডেটাতে প্রয়োগ করতে হবে। এতে Data Leakage হয় না।

ধাপ ৪: মিসিং ভ্যালু হ্যান্ডলিং

কাজ: ফাঁকা ঘর (NaN) পূরণ করা বা সরিয়ে ফেলা।

নিউমেরিক্যাল কলামের জন্য উপায়:

গড় (Mean) – সাধারণ, কিন্তু আউটলায়ার থাকলে সমস্যা
মাঝের মান (Median) – আউটলায়ার রেজিস্টেন্ট, সবচেয়ে সেফ
ধ্রুবক মান (Constant) – যেমন -১ বা ০ক্যাটেগরিক্যাল কলামের জন্য উপায়:
সবচেয়ে বেশি বার আসা মান (Mode)
নতুন ক্যাটাগরি তৈরি – যেমন "Unknown" বা "Missing"

ধাপ ৫: আউটলায়ার ডিটেকশন ও হ্যান্ডলিং

কাজ: স্বাভাবিকের চেয়ে চরম মানগুলো খুঁজে বের করে তাদের ঠিক করা।

উদাহরণ: বয়স কলামে ১৮০ বছর – স্পষ্ট ভুল।

কীভাবে হ্যান্ডেল করবেন?

ডিলিট করে দেওয়া
ক্যাপিং (একটা রেঞ্জের মধ্যে ফেলে আনা)
ট্রান্সফর্ম করা (লগ ট্রান্সফর্ম)

ধাপ ৬: এনকোডিং – ক্যাটেগরিকে সংখ্যায় রূপান্তর

কাজ: টেক্সট বা অবজেক্ট টাইপের কলামকে নম্বরে পরিণত করা। মডেল শুধু সংখ্যা বোঝে।

পদ্ধতিগুলো:

ওয়ান-হট এনকোডিং: প্রতিটি ক্যাটাগরির জন্য আলাদা কলাম (০ বা ১)। যেমন: city → Dhaka, Khulna, Rajshahi – তিনটি আলাদা কলাম।
অর্ডিনাল এনকোডিং: ক্যাটাগরির মধ্যে অর্ডার থাকলে (যেমন: শিক্ষাগত যোগ্যতা – প্রাইমারি, সেকেন্ডারি, গ্র্যাজুয়েট)
লেবেল এনকোডিং: যেখানে অর্ডার নেই, শুধু লেবেল পরিবর্তন (যেমন: রং – লাল, নীল, সবুজ → ০,১,২)

কখন কী ব্যবহার করবেন?

অর্ডার থাকলে → অর্ডিনাল

অর্ডার না থাকলে → ওয়ান-হট (লেবেল এনকোডিং বিভ্রান্তি তৈরি করতে পারে)

ধাপ ৭: স্কেলিং – নিউমেরিক্যাল মান সমান করা

কাজ: বড় মানের ফিচারকে ছোট মানের ফিচারের ওপর অযাচিত প্রভাব ফেলতে না দেওয়া।

উদাহরণ (বেতন ও সিজিপিএ):

সিজিপিএ রেঞ্জ: ১.০ – ৪.০
স্যালারি রেঞ্জ: ২০,০০০ – ১,০০,০০০ মডেল ভাববে স্যালারি বেশি গুরুত্বপূর্ণ, কারণ সংখ্যা বড়। স্কেলিং এই সমস্যা সমাধান করে।জনপ্রিয় পদ্ধতি:
মিন-ম্যাক্স স্কেলিং: সব মান ০ থেকে ১ এর মধ্যে নিয়ে আসে
স্ট্যান্ডার্ডাইজেশন (Z-score): গড় ০, স্ট্যান্ডার্ড ডেভিয়েশন ১ করে

ধাপ ৮ (অ্যাডভান্সড): ফিচার ক্রিয়েশন ও এক্সট্রাকশন

কাজ: নতুন কলাম তৈরি করে দেওয়া যা টার্গেট প্রেডিক্ট করতে সাহায্য করে।

উদাহরণ (টাইটানিক):

Name থেকে Title বের করা (Mr, Mrs, Master) – এটা নতুন ফিচার।
FamilySize = SibSp + Parch – বাঁচার সম্ভাবনা বাড়ায়।
AgeGroup বানিয়ে দেওয়া (Child, Young, Adult, Senior)

এটা অভিজ্ঞতা ও ডোমেইন নলেজের ব্যাপার। সময়ের সাথে শিখতে হবে।

Feature Engineering Starter

ধাপ ০: লাইব্রেরি ও ডেটা লোড করা

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

df = pd.read_csv('titanic_data_updated.csv')
df
```

pandas – ডেটা ম্যানিপুলেশনের জন্য
numpy – সংখ্যা নিয়ে কাজ করার জন্য
matplotlib ও seaborn – ভিজুয়ালাইজেশনের জন্য
train_test_split – ডেটা ট্রেন ও টেস্টে ভাগ করার জন্য
SimpleImputer – মিসিং ভ্যালু পূরণ করার জন্য
df – টাইটানিক ডেটাসেট লোড করা হয়েছে

ধাপ ১: অপ্রয়োজনীয় কলাম বাদ দেওয়া

কেন বাদ দেবেন?

কিছু কলাম টার্গেট (Survived) এর সাথে কোনো সম্পর্ক রাখে না। এগুলো থাকলে মডেল বিভ্রান্ত হয় এবং বাড়তি মেমরি খরচ হয়।

কোন কলামগুলো বাদ দেওয়া যায়?

কলাম	কেন বাদ দেবেন?
PassengerId	শুধু ক্রমিক নম্বর, বাঁচার সাথে সম্পর্ক নেই
Name	নাম দেখে কেউ বাঁচবে কিনা বোঝা যায় না
Ticket	এলোমেলো টিকেট নম্বর, কোনো প্যাটার্ন নেই
Cabin	অধিকাংশ ডেটাই মিসিং (687টির বেশি ফাঁকা), তাই বাদ দেওয়া ভালো

কোড:

```
df.drop(['PassengerId','Name','Ticket'], axis=1, inplace=True)
```

drop() – কলাম বা রো ডিলিট করে
['PassengerId','Name','Ticket'] – যে কলামগুলো ডিলিট করতে চাই
axis=1 – মানে কলাম বাদ দিচ্ছি (axis=0 মানে রো)
inplace=True – মূল ডেটাফ্রেমেই পরিবর্তন হবে (নতুন ভেরিয়েবল তৈরি না করে)

কেবিন কলাম বাদ দেওয়া (মিসিং ভ্যালুর কারণে)

```
df.isnull().sum()    # প্রতিটি কলামে কতটি মিসিং ভ্যালু আছে দেখায়
```

আউটপুট (মনে করুন):

```
Cabin    687
Age      177
...
```

Cabin এ 687টি মিসিং – যা মোট ডেটার প্রায় 77% (মোট ডেটা 891)। এত বড় অংশ পূরণ করা ঠিক না, তাই বাদ দেওয়া ভালো।

```
df.drop(['Cabin'], axis=1, inplace=True)
```

অ্যাডভান্স টিপ: কোনো কলামে 50% এর বেশি মিসিং থাকলে সাধারণত বাদ দেওয়া ভালো। কম থাকলে ইম্পিউট করা যায়।

বাদ দেওয়ার পর ডেটা দেখা:

```
df   # এখন PassengerId, Name, Ticket, Cabin চলে গেছে
```

এখন ডেটাফ্রেমে শুধু প্রাসঙ্গিক কলামগুলো আছে।

ধাপ ২: ফিচার (X) ও টার্গেট (y) আলাদা করা

কেন আলাদা করবেন?

মেশিন লার্নিং মডেল ইনপুট (X) ও আউটপুট (y) আলাদাভাবে নেয়। ট্রেনিং করার সময় একসাথে থাকলে কনফিউশন হয়।

কোড ও ব্যাখ্যা:

```
X = df.drop(['Survived'], axis=1)


y = df['Survived']
```

X → ইনপুট ফিচার। Survived কলাম বাদ দিয়ে বাকি সব কলাম।
y → টার্গেট (আউটপুট)। শুধু Survived কলাম।

X ও y দেখুন:

```
X   # Survived ছাড়া বাকি সব কলাম
y   # শুধু Survived কলাম (0 বা 1 মান)
```

উদাহরণ:

X এ থাকবে – Pclass, Sex, Age, SibSp, Parch, Fare, Embarked

y এ থাকবে – 0 (বাঁচেনি) বা 1 (বাঁচে)

গুরুত্বপূর্ণ কনসেপ্ট

axis=1 ও inplace=True কী?

axis=1 – কলাম বরাবর কাজ করে। কলাম ডিলিট করতে চাইলে axis=1 দিতে হবে।
inplace=True – মূল ডেটাফ্রেমেই পরিবর্তন করে। যদি inplace=False (ডিফল্ট) হয়, তাহলে নতুন ডেটাফ্রেম রিটার্ন করে, পুরোনো অপরিবর্তিত থাকে।

উদাহরণ:

```
# পদ্ধতি ১ (inplace=True)
df.drop(['PassengerId'], axis=1, inplace=True)  # df সরাসরি বদলে যাবে

# পদ্ধতি ২ (inplace=False)
df_new = df.drop(['PassengerId'], axis=1)  # df আগের মতো থাকবে, df_new এ পরিবর্তন
```

কেন মিসিং ভ্যালু বেশি হলে বাদ দেওয়া ভালো?

ধরুন, আপনার কাছে ১০০ জনের তথ্য আছে। Cabin কলামে ৭৭ জনের তথ্য নেই। বাকি ২৩ জনের কেবিন নম্বর থেকে কোনো প্যাটার্ন বের করা খুব কঠিন। ওই ২৩ জনের ডেটা দিয়ে বাকি ৭৭ জন পূরণ করলে ভুল হওয়ার সম্ভাবনা বেশি। তাই বাদ দেওয়া নিরাপদ।

ফিচার সিলেকশনের আরও পদ্ধতি (অ্যাডভান্স)

যা দেখানো হয়েছে তা ম্যানুয়াল ফিচার সিলেকশন। আরও পদ্ধতি আছে:

করিলেশন ম্যাট্রিক্স – ফিচার ও টার্গেটের সম্পর্ক দেখে
ভ্যারিয়েন্স থ্রেশহোল্ড – যেসব ফিচারে খুব কম পরিবর্তন আছে (যেমন সব মান প্রায় এক) সেগুলো বাদ দেওয়া
RFECV (Recursive Feature Elimination) – মডেল ব্যবহার করে অটো ফিচার সিলেকশন

তবে শুরুতে এই ম্যানুয়াল পদ্ধতিই যথেষ্ট।

Train Test Split Theory

ট্রেন-টেস্ট স্প্লিট কী? (Train-Test Split)

আপনার কাছে মোট ডেটা আছে। আপনি সেই ডেটাকে দুই ভাগে ভাগ করবেন:

ট্রেনিং ডেটা **(Train)** – এটা দিয়ে মডেল শেখাবে (প্যাটার্ন বুঝবে)
টেস্টিং ডেটা **(Test)** – এটা দিয়ে মডেল পরীক্ষা করবেন (কতটুকু সঠিক শিখেছে)

কেন ট্রেন-টেস্ট স্প্লিট প্রয়োজন?

ধরুন, আপনি একজন শিক্ষক। আপনার একজন ছাত্র আছে।

স্টেপ ১: পড়ানো (ট্রেনিং)

আপনি ছাত্রকে কিছু সাজেশন দিলেন। সে সেগুলো পড়ল, প্যাটার্ন বুঝল, শিখল।

স্টেপ ২: পরীক্ষা নেওয়া (টেস্টিং)

এখন আপনি দুটি ভিন্ন ধরনের টেস্ট নিতে পারেন:

টেস্টের ধরন	কী হয়?	ফলাফল
টেস্ট-১ (ভালো টেস্ট)	সাজেশনে না দেওয়া নতুন প্রশ্ন	ছাত্র সত্যিই শিখেছে কিনা বোঝা যায়
টেস্ট-২ (খারাপ টেস্ট)	সাজেশনের প্রশ্ন হুবহু দেওয়া	ছাত্র মুখস্থ করলেও ভালো নম্বর পাবে

আসল পারফরম্যান্স জানতে চাইলে নতুন প্রশ্ন দিতে হবে। ট্রেন-টেস্ট স্প্লিটও ঠিক একই কাজ করে।

মেশিন লার্নিংয়ে কীভাবে কাজ করে?

ধাপ ১: ডেটা ভাগ করা

মনে করুন, আপনার কাছে ১০০টি ডেটা পয়েন্ট আছে।

```
1মোট ডেটা = ১০০টি

2├── ট্রেনিং ডেটা = ৮০টি (৮০%) → মডেল শেখার জন্য

3└── টেস্টিং ডেটা = ২০টি (২০%) → মডেল পরীক্ষার জন্য
```

ধাপ ২: ট্রেনিং

মডেল শুধু ট্রেনিং ডেটা দেখে। এখান থেকে সে প্যাটার্ন শেখে।

ধাপ ৩: টেস্টিং

মডেল টেস্টিং ডেটা দেখে (যা সে আগে কখনো দেখেনি)। তারপর আমরা দেখি – মডেল কতটুকু সঠিক প্রেডিক্ট করতে পারে।

ধাপ ৪: অ্যাকিউরেসি বের করা

টেস্টিং ডেটার প্রেডিকশন আর আসল মানের সাথে মিলিয়ে অ্যাকিউরেসি (সঠিকতার হার) বের করি।

খারাপ পদ্ধতি কী?

ভুল পদ্ধতি: পুরো ডেটা দিয়েই ট্রেনিং করা, তারপর সেই একই ডেটা দিয়ে টেস্ট করা।

একে বলে **Data Leakage (**ডেটা লিকেজ**)** – তখন মডেল প্রশ্নগুলো মুখস্থ করে ফেলে। নতুন ডেটায় এসে তার পারফরম্যান্স খারাপ হয়।

উদাহরণ:

> ছাত্রকে সাজেশনের প্রশ্ন দিয়ে পড়ানো
> তারপর সেই একই প্রশ্ন দিয়ে পরীক্ষা নেওয়া
> ছাত্র ১০০% নম্বর পাবে, কিন্তু নতুন প্রশ্ন এলে ফেল করবে

মূলমন্ত্র**:** মডেল যেন টেস্টিং ডেটা কখনো দেখতে না পায় – শুধু ট্রেনিং ডেটা দেখবে।

ভাগ করার অনুপাত (কত শতাংশ কত হবে?)

সাধারণ নিয়ম:

ডেটার পরিমাণ	ট্রেনিং %	টেস্টিং %
বড় ডেটাসেট (লাখের বেশি)	৯৮-৯৯%	১-২%
মাঝারি ডেটাসেট (হাজার)	৮০-৯০%	১০-২০%
ছোট ডেটাসেট (শতাধিক)	৭০-৮০%	২০-৩০%

ডিফল্ট সুবিধার অনুপাত**:** ৮০% ট্রেন, ২০% টেস্ট (বেশিরভাগ ক্ষেত্রেই কাজ করে)

গুরুত্বপূর্ণ কনসেপ্ট

১. কেন ৫০-৫০ ভাগ করি না?

ট্রেনিং ডেটা কম হলে মডেল ঠিকমত শিখতে পারে না। আবার টেস্টিং ডেটা কম হলে মডেলের আসল পারফরম্যান্স বোঝা যায় না। ৮০-২০ বা ৭০-৩০ একটা ভালো ব্যালেন্স।

২. স্ট্রেটিফাইড স্প্লিট কী?

ধরুন, আপনার ডেটাতে Survived = 0 এবং 1 এর সংখ্যা খুবই অসম। ১০০টি ডেটার মধ্যে ৯০টি বাঁচেনি (০), ১০টি বাঁচে (১)।

সাধারণ স্প্লিট করলে হতে পারে ট্রেনিং এ সব ০ চলে যাবে, টেস্টিং এ ১। তখন মডেল ১ দেখা ছাড়াই টেস্ট দেবে – ফল খারাপ হবে।

স্ট্রেটিফাইড স্প্লিট নিশ্চিত করে যে ট্রেন ও টেস্ট – উভয় অংশেই ০ ও ১ এর অনুপাত মূল ডেটার মতো থাকবে।

৩. Random State কেন?

```
train_test_split(X, y, test_size=0.2, random_state=42)
```

random_state নির্দিষ্ট করে দিলে প্রতি বার একই ভাবে ডেটা ভাগ হবে। না দিলে প্রতি বার আলাদা ভাবে ভাগ হয়।

গবেষণার জন্য random_state ফিক্স করে দেওয়া ভালো – যেন রেজাল্ট রিপ্রোডিউসেবল হয়।

Train_Test Split Code

ট্রেন-টেস্ট স্প্লিট – হাতে-কলমে

আগের আমরা শিখেছি:

- কেন ট্রেন-টেস্ট স্প্লিট করতে হয়
- ৮০-২০ বা ৭০-৩০ ভাগে ডেটা ভাগ করা হয়

আমরা দেখবো কীভাবে কোড দিয়ে স্প্লিট করবেন।

ধাপ ১: ডেটা প্রস্তুত (আগে যা করেছি)

আমাদের কাছে এখন দুটি জিনিস আছে:

```
X = df.drop(['Survived'], axis=1)    # ফিচার কলাম (ইনপুট)

y = df['Survived']                   # টার্গেট কলাম (আউটপুট)
```

মনে রাখার ছবি:

```
আসল ডেটা (891টি রো)
┌──────────────────────────────────────────────────┐
│  X (ফিচার)                  |  y (টার্গেট)      │
│  Pclass, Sex, Age, ...      |  Survived (0/1)    │
└──────────────────────────────────────────────────┘
```

ধাপ ২: train_test_split ইম্পোর্ট করা

```
from sklearn.model_selection import train_test_split
```

sklearn.model_selection মডিউলের ভেতরে train_test_split নামে একটি ফাংশন আছে যা ডেটা ভাগ করার কাজ করে।

ধাপ ৩: স্প্লিট করা

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

কোডটি লাইন বাই লাইন ব্যাখ্যা:

অংশ	কী বোঝায়?
X_train, X_test, y_train, y_test	আউটপুট ভেরিয়েবল – চারটা জিনিস রিটার্ন করে
train_test_split(...)	ফাংশন কল
X, y	ইনপুট – ফিচার আর টার্গেট
test_size=0.2	টেস্টিং সেটের সাইজ ২০% (বাকি ৮০% ট্রেনিং)
random_state=42	রেন্ডম স্টেট – প্রতি বার একই ভাবে ভাগ হবে

আউটপুট কী কী পাবেন?

আউটপুট ভেরিয়েবল	কী থাকে?	কতটি ডেটা? (891 এর মধ্যে)
X_train	ট্রেনিং-এর ফিচার (ইনপুট)	712টি (80%)
X_test	টেস্টিং-এর ফিচার (ইনপুট)	179টি (20%)
y_train	ট্রেনিং-এর টার্গেট (আউটপুট)	712টি (80%)
y_test	টেস্টিং-এর টার্গেট (আউটপুট)	179টি (20%)

ভিজুয়াল ধারণা (কীভাবে ডেটা ভাগ হয়)

আপনার আসল X এবং y – দুটোই সারিবদ্ধভাবে আছে।

```
ইনডেক্স     X (ফিচার)                y (টার্গেট)

0           [Pclass, Sex, Age...] → 0 (বাঁচেনি)

1           [Pclass, Sex, Age...] → 1 (বাঁচে)

2           [Pclass, Sex, Age...] → 1 (বাঁচে)

3           [Pclass, Sex, Age...] → 0 (বাঁচেনি)

...

890         [Pclass, Sex, Age...] → 0 (বাঁচেনি)
```

train_test_split করার পর:

```
X_train (712টি) ⟷ y_train (712টি)  [একই ইন্ডেক্সের জোড়া]

X_test  (179টি) ⟷ y_test  (179টি)  [একই ইন্ডেক্সের জোড়া]
```

গুরুত্বপূর্ণ: যেই ইন্ডেক্সের ডেটা X_train এ যাবে, সেই ইন্ডেক্সের y এর মান y_train এ যাবে। সম্পর্কটা নষ্ট হয় না।

ধাপ ৪: ফলাফল যাচাই করা

X_train দেখা:

```
display(X_train)
```

ফলাফল: 712টি রো, ৮টি কলাম (Survived বাদে সব ফিচার)

y_train দেখা:

```
display(y_train)
```

ফলাফল: 712টি মান (0 বা 1)

X_test দেখা:

```
X_test
```

ফলাফল: 179টি রো

y_test দেখা:

```
display(y_test)
```

ফলাফল: 179টি মান (0 বা 1)

গুরুত্বপূর্ণ বিষয়: ইন্ডেক্সের মিল

লক্ষ্য করুন, X_train এবং y_train এ একই ইন্ডেক্স সংখ্যা আছে:

```
X_train.head()  # ইন্ডেক্স দেখাবে: 331, 373, 131, 740, 27
```

```
y_train.head()  # ইন্ডেক্স দেখাবে: 331, 373, 131, 740, 27
```

অর্থাৎ: X_train এর প্রথম ডেটা (ইন্ডেক্স 331) – তার টার্গেট মান y_train এর প্রথম মান (ইন্ডেক্স 331) এ আছে। সম্পর্ক নষ্ট হয়নি।

random_state = 42 কেন?

random_state ঠিক করে দিলে:

আপনি যা করবেন	ফলাফল
প্রথমবার রান করবেন	নির্দিষ্ট 712টি ডেটা ট্রেনিং এ যাবে
দ্বিতীয়বার রান করবেন	ঠিক একই 712টি ডেটা ট্রেনিং এ যাবে
random_state বাদ দিলে	প্রতি বার ভিন্ন ভিন্ন ডেটা ট্রেনিং এ যাবে

42 কেন?

এটা একধরনের কনভেনশন (কমিক্যাল রেফারেন্স: "হিচহাইকার্স গাইড টু দ্যা গ্যালাক্সি" বইতে 42 উত্তর হিসেবে আছে)। আপনি 0, 1, 100 – যেকোনো সংখ্যা দিতে পারেন। শুধু মনে রাখবেন একই সংখ্যা দিলে একই ফল পাবেন।

পুরো প্রক্রিয়া এক নজরে

```
# 1. লাইব্রেরি ইম্পোর্ট
from sklearn.model_selection import train_test_split

```

```
# 2. আগে থেকেই X ও y ডিফাইন করা আছে
# X = df.drop(['Survived'], axis=1)
# y = df['Survived']

# 3. ট্রেন-টেস্ট স্প্লিট
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,       # 20% টেস্ট, 80% ট্রেন
    random_state=42      # রিপ্রোডিউসিবিলিটির জন্য
)

# 4. ভেরিফাই করা
print(f"X_train shape: {X_train.shape}")  # (712, 8)
print(f"X_test shape: {X_test.shape}")    # (179, 8)
print(f"y_train shape: {y_train.shape}")  # (712,)
print(f"y_test shape: {y_test.shape}")    # (179,)
```

Handling Missing Values

মিসিং ভ্যালু ইম্পিউটেশন কী?

ডেটাসেটের ফাঁকা ঘরগুলো (NaN, null) পূরণ করাকে ইম্পিউটেশন বলে। মেশিন লার্নিং মডেল ফাঁকা জায়গা বুঝতে পারে না, তাই এগুলো পূরণ করা জরুরি।

টাইটানিক ডেটাসেটে মিসিং ভ্যালু কোথায় ছিল?

```
df.isnull().sum()
```

ফলাফল:

কলাম	মিসিং ভ্যালুর সংখ্যা	টাইপ
Age	177টি	নিউমেরিক্যাল (বয়স)
Cabin	687টি	ক্যাটাগরিক্যাল (কেবিন নম্বর)
Embarked	2টি	ক্যাটাগরিক্যাল (যাত্রা শুরুর বন্দর)

মোট ডেটা: 891টি স্যাম্পল

মিসিং ভ্যালু হ্যান্ডলিংয়ের পদ্ধতি

পদ্ধতি ১: ডিলিট করা (Deletion)

ক) রো ডিলিট করা (Row Deletion)

যেসব রোতে মিসিং ভ্যালু আছে, পুরো রো মুছে ফেলা।

উদাহরণ: কোনো যাত্রীর বয়স নেই → পুরো যাত্রীর তথ্য বাদ দেওয়া।

সমস্যা:

Cabin কলামে 687টি রোতে মিসিং থাকলে 687টি রো ডিলিট করলে ডেটা প্রায় শেষ (বাকি থাকে 891-687 = 204টি মাত্র)।

মনে রাখবেন: অনেক ডেটা ডিলিট করলে মডেল ঠিকমত শিখতে পারে না।

খ) কলাম ডিলিট করা (Column Deletion)

পুরো কলাম মুছে ফেলা।

কখন করবেন?

একটি কলামে অনেক বেশি (সাধারণত 50%+) মিসিং থাকলে।

টাইটানিকের উদাহরণ:

> Cabin কলামে 687টি মিসিং → মোটের 77% মিসিং
> এত বড় অংশ পূরণ করা কঠিন, বরং কলাম বাদ দেওয়াই ভালো

```
1df.drop(['Cabin'], axis=1, inplace=True)
```

পদ্ধতি ২: ইম্পিউট করা (Imputation) – ফাঁকা জায়গা পূরণ করা

এটাই প্রধান ও ফিজিবল সলিউশন। ডেটা টাইপ অনুযায়ী পদ্ধতি আলাদা।

নিউমেরিক্যাল ডেটার ইম্পিউটেশন

Age কলামে 177টি মিসিং আছে (মোটের 20% মাত্র)। এগুলো পূরণ করা যায়।

দুটি প্রধান পদ্ধতি:

পদ্ধতি	কী করে?	কখন ব্যবহার করবেন?
মিন (Mean)	গড় মান বসায়	ডেটা নরমালি ডিস্ট্রিবিউটেড থাকলে (বেল-শেপ আকৃতি)
মিডিয়ান (Median)	মাঝের মান বসায়	ডেটাতে আউটলায়ার থাকলে বা স্কিউড হলে

মিন ও মিডিয়ানের উদাহরণ:

ধরুন Age এর কিছু মান: [22, 24, 25, 28, 30, 100]

```
1মিন (Mean) = (22+24+25+28+30+100) ÷ 6 = 38.17
```

```
2মিডিয়ান (Median) = (25+28) ÷ 2 = 26.5  (ছোট থেকে সাজালে মাঝের দুটির গড়)
```

আউটলায়ার 100 থাকায় মিন বেড়ে গেছে (38.17), মিডিয়ান ঠিক আছে (26.5)।

Age-এর মতো ডেটায় সাধারণত মিডিয়ান বেশি নিরাপদ।

ভিজুয়াল ধারণা:

```
1নরমাল ডিস্ট্রিবিউশন (বেল শেপ)      স্কিউড ডিস্ট্রিবিউশন (ডান দিকে লেজ)
2         /\                                  /\
3        /  \                                /  \____
4       /    \                              /
5     ________                            _________
6     Mean = Median                       Mean > Median
7
8→ Mean ব্যবহার করা যায়              → Median ব্যবহার করুন
```

বেস্ট প্র্যাকটিস: Age-এর মতো ক্ষেত্রে Median ব্যবহার করুন – কারণ বয়সের ডেটায় সাধারণত আউটলায়ার থাকে (যেমন 100 বছর)।

ক্যাটাগরিক্যাল ডেটার ইম্পিউটেশন

Embarked-এ মাত্র 2টি মিসিং। ক্যাটাগরিক্যাল ডেটার জন্য পদ্ধতি:

পদ্ধতি	কী করে?	উদাহরণ
মোড (Mode)	সবচেয়ে বেশি বার আসা ক্যাটাগরি বসায়	Embarked-এ 'S' যদি 600 বার আসে, 'C' আসে 200 বার, 'Q' আসে 89 বার → তাহলে 'S' বসাবেন
নতুন ক্যাটাগরি	"Unknown" বা "Missing" লেবেল বসায়	এতে মডেল বুঝতে পারে তথ্যটি নেই

মোড বের করার নিয়ম:

```
1mode_value = df['Embarked'].mode()[0]  # সবচেয়ে বেশি আসা মান
```

কেন মোড ব্যবহার করবেন?

ক্যাটাগরিক্যাল ডেটার গড় বা মাঝের মান নেই
সবচেয়ে কমন মান বসানো সবচেয়ে লজিক্যাল

গুরুত্বপূর্ণ সতর্কতা

1. ট্রেন-টেস্ট স্প্লিটের আগে ইম্পিউট করবেন না!

ভুল পদ্ধতি:

```
# ভুল
imputer.fit(X)            # পুরো X (ট্রেন+টেস্ট) দিয়ে শিখলাম
X_train, X_test = split   # তারপর স্প্লিট → লিকেজ!
```

সঠিক পদ্ধতি:

```
# সঠিক
X_train, X_test, y_train, y_test = train_test_split(X, y)
imputer.fit(X_train)      # শুধু ট্রেনিং ডেটা দিয়ে শিখবো
X_train = imputer.transform(X_train)
X_test = imputer.transform(X_test)    # টেস্টে ট্রান্সফর্ম মাত্র
```

2. "Missing" লেবেল কেন কাজে লাগে?

মাঝে মাঝে মিসিং থাকাটাই একটি ইনফরমেশন হতে পারে। যেমন: কারো কেবিন নম্বর নেই – হয়তো সে ইকোনমি ক্লাসের যাত্রী। নতুন ক্যাটাগরি "Missing" দিয়ে মডেল নিজেই শিখতে পারে এটা।

3. কখন মিন বা মিডিয়ান?

ডেটার ধরন	ব্যবহার করুন	কারণ
নরমাল ডিস্ট্রিবিউশন (বেল শেপ)	**মিন**	গড় মান সবচেয়ে প্রতিনিধিত্বমূলক
স্কিউড ডিস্ট্রিবিউশন (লম্বা লেজ)	**মিডিয়ান**	আউটলায়ারের প্রভাব কম
ডেটাতে বড় আউটলায়ার আছে	**মিডিয়ান**	নিরাপদ
ইনকাম, বয়স, প্রাইস	**মিডিয়ান**	সাধারণত স্কিউড থাকে

Numerical Missing value Imputation using Pandas

নিউমেরিক্যাল মিসিং ভ্যালু ইম্পিউটেশন – প্যান্ডাস

Age কলামে (নিউমেরিক্যাল) মিসিং ভ্যালু কিভাবে পূরণ করবেন
মিন **(Mean)** ও মিডিয়ান **(Median)** – দুই পদ্ধতিতেই ইম্পিউটেশন
ট্রেনিং ও টেস্ট ডেটা আলাদাভাবে হ্যান্ডল করা
KDE plot দিয়ে ইম্পিউটেশনের প্রভাব দেখা

ধাপ ১: ট্রেন ও টেস্ট ডেটায় মিসিং চেক করা

আগের আমরা X_train, X_test, y_train, y_test স্প্লিট করেছিলাম।

কোড:

```
X_train.isnull().sum()
```

```
X_test.isnull().sum()
```

ফলাফল:

X_train এ **(712**টি ডেটা**):**

কলাম	মিসিং সংখ্যা
Age	140টি
অন্যান্য কলাম	0টি

X_test এ **(179**টি ডেটা**):**

কলাম	মিসিং সংখ্যা
Age	37টি
অন্যান্য কলাম	0টি

মনে রাখবেন**:** ট্রেন ও টেস্ট – উভয় স্থানেই Age কলামে মিসিং আছে। কিন্তু আমরা শুধু ট্রেনিং ডেটা থেকে শিখবো, টেস্ট ডেটা থেকে নয়।

ধাপ ২: মূল নিয়ম – টেস্ট ডেটা থেকে কখনো শিখবেন না

```
✅ সঠিক পদ্ধতি:
   1. X_train থেকে mean/median বের করুন
   2. X_train এর মিসিং ভ্যালু পূরণ করুন
   3. X_test এর মিসিং ভ্যালু পূরণ করুন (ট্রেনের mean/median ব্যবহার করে)
```

```
5
```

```
6❌ ভুল পদ্ধতি:
7   1. পুরো X (ট্রেন+টেস্ট) থেকে mean বের করা
8   2. টেস্ট ডেটার mean দিয়ে ট্রেন পূরণ করা
```

কেন এত গুরুত্বপূর্ণ?

টেস্ট ডেটা আসলে অদেখা ডেটা। মডেল টেস্ট করার সময় টেস্ট ডেটা প্রথমবার দেখবে। যদি টেস্ট ডেটা থেকে কোনো পরিসংখ্যান (mean) শিখে ফেলি, তাহলে ডেটা লিকেজ (Data Leakage) হয় – মডেলের আসল পারফরম্যান্স বোঝা যায় না।

ধাপ ৩: প্যান্ডাস দিয়ে মিন ইম্পিউটেশন

৩.১: ট্রেনিং ডেটা থেকে Mean বের করা

```
1age_mean = X_train['Age'].mean()
```

```
2print(age_mean)
```

X_train এর Age কলামের সব বিদ্যমান মানের গড় বের করলাম। মিসিং ভ্যালুগুলো এই গড় মান দিয়ে পূরণ করবো।

৩.২: ট্রেনিং ডেটায় নতুন কলাম তৈরি করে Mean দিয়ে ইম্পিউট করা

```
1X_train['age_mean_imputor'] = X_train['Age'].fillna(age_mean)
```

X_train['Age'] – আসল Age কলাম
.fillna(age_mean) – যেখানে NaN আছে, সেখানে age_mean বসাও
X_train['age_mean_imputor'] – নতুন কলাম তৈরি করলাম (আসল কলাম অপরিবর্তিত রেখে)

৩.৩: টেস্টিং ডেটায় একই Mean দিয়ে ইম্পিউট করা

```
1X_test['age_mean_imputor'] = X_test['Age'].fillna(age_mean)
```

গুরুত্বপূর্ণ: টেস্টের জন্য নতুন করে mean বের করছি না। ট্রেনের mean ব্যবহার করছি।

কেন নতুন কলাম বানাবেন?

২টি কারণে নতুন কলাম বানানো হয়েছে:

আসল Age কলাম অপরিবর্তিত রাখতে (পরে compare করার জন্য)
Mean ও Median ইম্পিউটেশন – দুইটার ফলাফল পাশাপাশি দেখতে
KDE plot এ দুটোর ডিস্ট্রিবিউশন তুলনা করতে

প্রোডাকশনে আপনি চাইলে সরাসরি **Age** কলামই আপডেট করতে পারেন:

```
1X_train['Age'] = X_train['Age'].fillna(age_mean)  # সরাসরি পরিবর্তন
```

ধাপ ৪: প্যান্ডাস দিয়ে মিডিয়ান ইম্পিউটেশন (Median Imputation)

৪.১: ট্রেনিং ডেটা থেকে Median বের করা

```
age_median = X_train['Age'].median()
```

```
print(age_median)
```

৪.২: ট্রেনিং ডেটায় Median দিয়ে ইম্পিউট করা

```
X_train['age_median_imputor'] = X_train['Age'].fillna(age_median)
```

৪.৩: টেস্টিং ডেটায় একই Median দিয়ে ইম্পিউট করা

```
X_test['age_median_imputor'] = X_test['Age'].fillna(age_median)
```

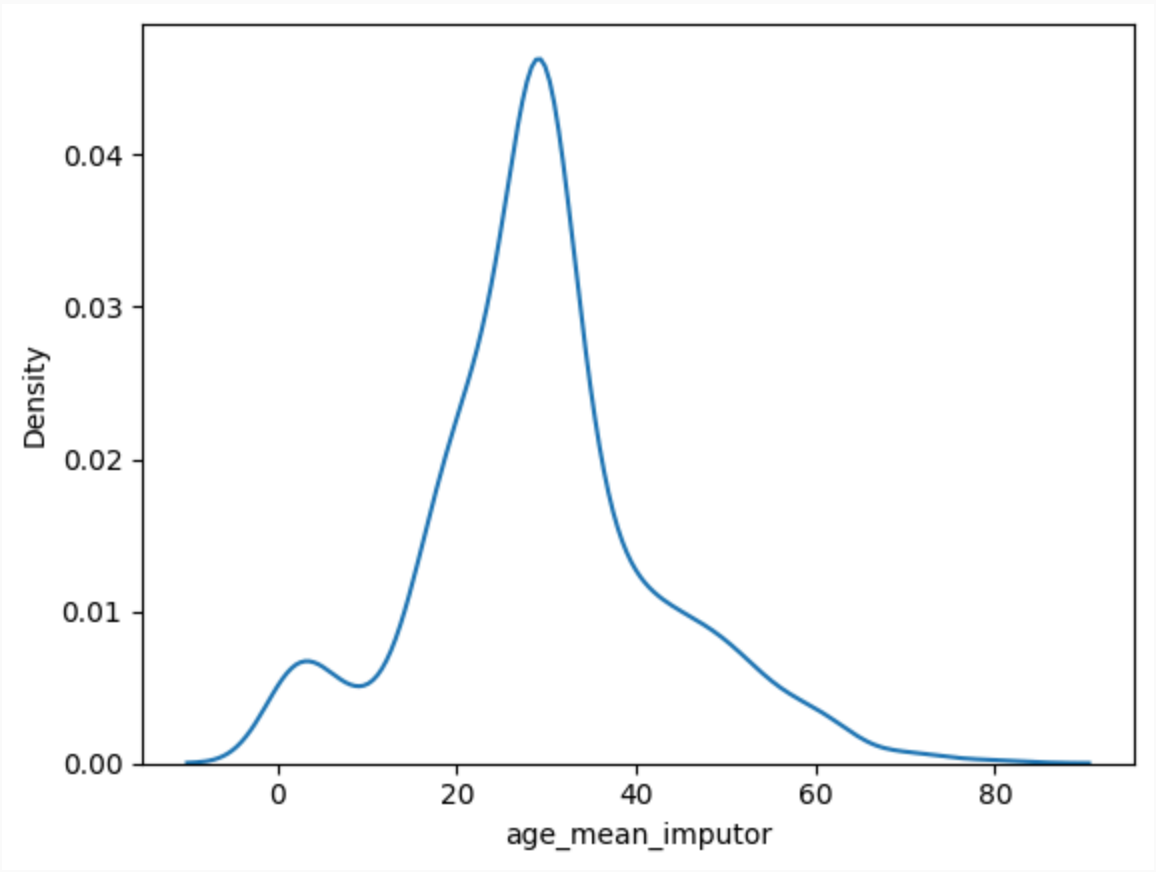
ধাপ ৫: ভিজুয়ালাইজেশন – KDE Plot দিয়ে ইম্পিউটেশনের প্রভাব দেখা

KDE (Kernel Density Estimate) plot দেখায় ডেটার ডিস্ট্রিবিউশন কেমন।

মিন ইম্পিউটেশনের KDE plot:

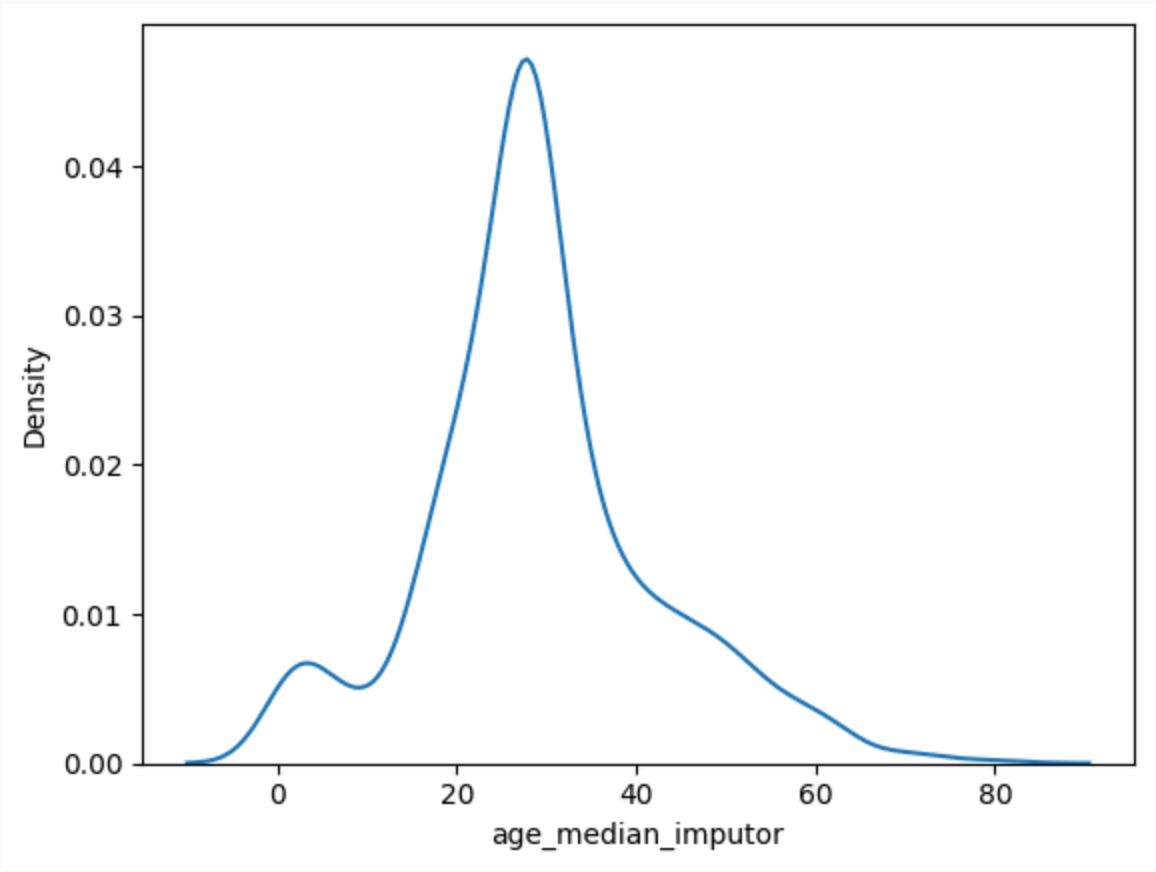
```
import seaborn as sns
```

```
sns.kdeplot(data=X_train, x='age_mean_imputor')
```



মিডিয়ান ইম্পিউটেশনের KDE plot:

```
sns.kdeplot(data=X_train, x='age_median_imputor')
```



যা দেখা গেল:

ইম্পিউটেশন পদ্ধতি	KDE plot এ প্রভাব
Mean ইম্পিউটেশন	ডিস্ট্রিবিউশনের একটা অংশ উপরের দিকে উঠে গেছে (standard deviation কমেছে)
Median ইম্পিউটেশন	প্রায় same রকম ডিস্ট্রিবিউশন

টাইটানিকের Age ডেটার জন্য Mean ও Median – উভয় পদ্ধতিই প্রায় কাছাকাছি ফলাফল দিয়েছে। যেকোনো একটি ব্যবহার করা যাবে।

Mean vs Median – কখন কোনটা ব্যবহার করবেন?

পদ্ধতি	সূত্র	কখন ব্যবহার করবেন	উদাহরণ
Mean	(সকল মানের যোগফল) / (মোট মানের সংখ্যা)	ডেটা **নরমাল ডিস্ট্রিবিউটেড** হলে (বেল শেপ)	উচ্চতা, ওজন (সাধারণত)
Median	ছোট থেকে সাজালে মাঝের মান	ডেটাতে **আউটলায়ার** থাকলে বা **স্কিউড** হলে	বেতন, বয়স, বাড়ির দাম

টাইটানিকের Age ডেটার জন্য কোনটা ভালো?

Age ডেটাতে সাধারণত কিছু আউটলায়ার থাকে (যেমন 80-100 বছর)। তাই **Median** নিরাপদ পছন্দ। তবে দেখা গেছে, টাইটানিকের Age ডেটায় Mean ও Median খুব বেশি আলাদা নয়।

পরীক্ষার উপায়:

```
# Mean এবং Median এর পার্থক্য দেখুন
print(f"Mean: {age_mean}")
print(f"Median: {age_median}")
print(f"Difference: {abs(age_mean - age_median)}")
```

পার্থক্য কম হলে → যেকোনো একটা চলবে
পার্থক্য বেশি হলে → Median ব্যবহার করুন

পুরো কোড একসাথে

```
# 1. মিসিং চেক
X_train.isnull().sum()
X_test.isnull().sum()

# 2. Mean ইম্পিউটেশন
age_mean = X_train['Age'].mean()
X_train['age_mean_imputor'] = X_train['Age'].fillna(age_mean)
X_test['age_mean_imputor'] = X_test['Age'].fillna(age_mean)

# 3. Median ইম্পিউটেশন
age_median = X_train['Age'].median()
X_train['age_median_imputor'] = X_train['Age'].fillna(age_median)
X_test['age_median_imputor'] = X_test['Age'].fillna(age_median)

# 4. ভিজুয়ালাইজেশন
import seaborn as sns
sns.kdeplot(data=X_train, x='age_mean_imputor')
sns.kdeplot(data=X_train, x='age_median_imputor')
```

গুরুত্বপূর্ণ সতর্কতা

1. কেন Mean/Median ট্রেন থেকে বের করবেন?

```
1আপনার কাছে 100 টি ডেটা আছে: 80 টি ট্রেন, 20 টি টেস্ট।
2
3✅ সঠিক: ট্রেনের 80 টি ডেটা থেকে Mean বের করুন → ট্রেন ও টেস্ট উভয়ে ইম্পিউট করুন
4❌ ভুল: পুরো 100 টি ডেটা থেকে Mean বের করুন → তারপর ট্রেন-টেস্ট স্প্লিট করুন
```

2. Mean/Median ইম্পিউটেশনের সীমাবদ্ধতা

Mean ইম্পিউটেশন: ডেটার ভ্যারিয়েশন কমিয়ে দেয় (standard deviation কমে যায়)
Median ইম্পিউটেশন: তুলনামূলক ভালো, কিন্তু মিসিং অনেক বেশি হলে সমস্যা
অ্যাডভান্স পদ্ধতি: KNN Imputer, Iterative Imputer (পরবর্তী লেভেলে)

3. কখন নতুন কলাম বানাবেন, কখন সরাসরি আপডেট করবেন?

পরিস্থিতি	করবেন
এক্সপেরিমেন্ট/রিসার্চ	নতুন কলাম বানান (Mean/Median তুলনা করতে)
প্রোডাকশন/ফাইনাল মডেল	সরাসরি আপডেট করুন (X_train['Age'].fillna(age_mean, inplace=True))

Numerical Missing Values Imputation using Simple Imputer

সিম্পল ইম্পিউটার (SimpleImputer)

সিম্পল ইম্পিউটার কী?

Scikit-learn লাইব্রেরির একটি ক্লাস যা মিসিং ভ্যালু পূরণের কাজ করে। এটি মেশিন লার্নিং পাইপলাইনের অংশ হিসেবে ব্যবহার করা যায়।

প্যান্ডাস **vs** সিম্পল ইম্পিউটার:

প্যান্ডাস পদ্ধতি	সিম্পল ইম্পিউটার পদ্ধতি
দ্রুত, ছোট প্রজেক্টের জন্য	ইন্ডাস্ট্রি লেভেল, পাইপলাইন ফ্রেন্ডলি
ম্যানুয়ালি mean/median বের করতে হয়	ফিট() মেথড নিজে বের করে
ট্রেন-টেস্ট আলাদা করা কঠিন	ট্রেনিং এ ফিট(), টেস্ট এ ট্রান্সফর্ম() – ক্লিন সেপারেশন
মডেলের সাথে ইন্টিগ্রেট করা যায় না	স্কেলার, এনকোডার, মডেল – সব পাইপলাইনে যুক্ত করা যায়

ইন্ডাস্ট্রি প্র্যাকটিস: রিয়েল ওয়ার্ল্ড প্রজেক্টে সবসময় সিম্পল ইম্পিউটার (বা এর ভ্যারিয়েন্ট) ব্যবহার করা হয়।

ধাপ ১: সিম্পল ইম্পিউটার ইম্পোর্ট করা

```
from sklearn.impute import SimpleImputer
```

sklearn.impute মডিউলের ভেতরে SimpleImputer নামের ক্লাসটি আছে।

ধাপ ২: ইম্পিউটার অবজেক্ট তৈরি করা

```
age_imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
```

প্যারামিটারগুলোর ব্যাখ্যা:

প্যারামিটার	মান	ব্যাখ্যা
missing_values	np.nan	কী ধরনের মান মিসিং হিসেবে চিহ্নিত হবে (এখানে NaN)
strategy	'mean'	কী দিয়ে পূরণ করবে

strategy-এর সম্ভাব্য মান:

strategy	কী করে	কখন ব্যবহার করবেন
'mean'	গড় মান বসায়	নরমাল ডিস্ট্রিবিউশনের জন্য
'median'	মাঝের মান বসায়	স্কিউড ডেটা বা আউটলায়ার থাকলে
'most_frequent'	সবচেয়ে বেশি আসা মান (মোড) বসায়	ক্যাটাগরিক্যাল ডেটার জন্য
'constant'	নির্দিষ্ট একটি মান বসায়	fill_value প্যারামিটার দিয়ে মান দিতে হয়

মিডিয়ানের জন্য: strategy='median'

মোডের জন্য: strategy='most_frequent'

কনস্ট্যান্টের জন্য: strategy='constant', fill_value=0

ধাপ ৩: ফিট (Fit) – ট্রেনিং ডেটা থেকে শেখা

```
age_imputer.fit(X_train[['Age']])
```

ফিট কী করে?

```
আপনার বলা strategy = 'mean'
আমি X_train['Age'] কলামে যাবো
ওখানের বিদ্যমান মানগুলো দেখে গড় (mean) বের করবো
এই গড় মানটা মেমরিতে সংরক্ষণ করবো
```

মনে রাখবেন: ফিট শুধু শেখে, ডেটা পরিবর্তন করে না।

গুরুত্বপূর্ণ: ডেটা টু-ডি হতে হবে

```
# সঠিক
age_imputer.fit(X_train[['Age']])      # ডাবল ব্র্যাকেট → 2D

# ভুল
age_imputer.fit(X_train['Age'])        # সিঙ্গেল ব্র্যাকেট → 1D (error দিবে)
```

কেন **2D** দিতে হয়?

Scikit-learn-এর সব ট্রান্সফরমার ইনপুট হিসেবে (n_samples, n_features) আকৃতির 2D ডেটা আশা করে। একটি কলাম হলেও 2D করতে হবে।

ধাপ ৪: ট্রান্সফর্ম (Transform) – ট্রেনিং ডেটা পরিবর্তন করা

```
X_train['Age'] = age_imputer.transform(X_train[['Age']]).ravel()
```

এই লাইনে কী হলো?

অংশ	ব্যাখ্যা
age_imputer.transform(...)	ফিট-এ শেখা mean মান ব্যবহার করে মিসিং ভ্যালু পূরণ করে
X_train[['Age']]	2D আকারে Age কলাম ইনপুট দেওয়া
transform()-এর আউটপুট	2D array (যেমন: [[22], [25], [30]])
.ravel()	2D array-কে 1D তে রূপান্তর (যেমন: [22, 25, 30])
X_train['Age']	পুরোনো Age কলামের জায়গায় নতুন ইম্পিউটেড মান বসানো

রূপান্তরটি ভিজুয়ালি:

```
BEFORE (X_train['Age']):
    Age
0   22.0
1   NaN   ← মিসিং
2   26.0
3   NaN   ← মিসিং
4   35.0

AFTER transform + ravel:
     Age
0   22.0
1   29.5  ← mean=29.5 বসেছে
2   26.0
3   29.5  ← mean=29.5 বসেছে
4   35.0
```

ধাপ ৫: ট্রেনিং ডেটা যাচাই করা

```
X_train.isnull().sum()
```

ফলাফল: Age কলামে 0 মিসিং (সব পূরণ হয়ে গেছে)

ধাপ ৬: পুরোনো ইম্পিউটেড কলাম ডিলিট করা

আগের ভিডিওতে আমরা age_mean_imputor ও age_median_imputor নামে দুটি কলাম বানিয়েছিলাম। সেগুলো এখন দরকার নেই।

```
X_train.drop(['age_mean_imputor', 'age_median_imputor'], axis=1,
inplace=True)
```

কেন ডিলিট করলাম?

> ওগুলো এক্সপেরিমেন্টাল কলাম ছিল
> এখন আসল Age কলাম ইম্পিউটেড হয়ে গেছে
> বাড়তি কলাম রাখলে মডেল বিভ্রান্ত হতে পারে

ধাপ ৭: টেস্টিং ডেটা ট্রান্সফর্ম করা

```
X_test['Age'] = age_imputer.transform(X_test[['Age']])
```

গুরুত্বপূর্ণ: টেস্ট ডেটার জন্য ফিট করছি না – শুধু ট্রান্সফর্ম করছি।
ট্রেন থেকে শেখা mean মান ব্যবহার করে টেস্টের মিসিং ভ্যালু পূরণ হবে।

টেস্ট ডেটা যাচাই:

```
X_test.isnull().sum()
```

ফলাফল: Age কলামে 0 মিসিং

টেস্ট ডেটা থেকে পুরোনো কলাম ডিলিট:

```
X_test.drop(['age_mean_imputor', 'age_median_imputor'], axis=1,
inplace=True)
```

পুরো প্রক্রিয়া একসাথে (ভিডিওর কোড সংক্ষেপে)

```
# 1. ইম্পোর্ট
from sklearn.impute import SimpleImputer

# 2. ইম্পিউটার অবজেক্ট (mean দিয়ে)
age_imputer = SimpleImputer(missing_values=np.nan, strategy='mean')

# 3. ট্রেনিং ডেটা থেকে শেখা (ফিট)
age_imputer.fit(X_train[['Age']])

# 4. ট্রেনিং ডেটা ট্রান্সফর্ম
X_train['Age'] = age_imputer.transform(X_train[['Age']]).ravel()
```

```
12

13# 5. টেস্টিং ডেটা ট্রান্সফর্ম (শুধু ট্রান্সফর্ম, ফিট না)
14X_test['Age'] = age_imputer.transform(X_test[['Age']])
15

16# 6. পুরোনো এক্সপেরিমেন্টাল কলাম ডিলিট
17X_train.drop(['age_mean_imputor', 'age_median_imputor'], axis=1,
inplace=True)
18X_test.drop(['age_mean_imputor', 'age_median_imputor'], axis=1,
inplace=True)
```

fit(), transform(), fit_transform() – পার্থক্য বোঝা

মেথড	কী করে	কখন ব্যবহার করবেন
fit(X)	শুধু শেখে (যেমন mean বের করে), ডেটা পরিবর্তন করে না	ট্রেনিং ডেটা দিয়ে
transform(X)	আগে শেখা নিয়ম প্রয়োগ করে	ট্রেন ও টেস্ট – উভয়ে (ফিট করার পর)
fit_transform(X)	শেখে + প্রয়োগ করে – একসাথে	শুধু ট্রেনিং ডেটার জন্য (সুবিধার জন্য)

উদাহরণ:

```
1# পদ্ধতি ১ (আমরা করেছি)
2imputer.fit(X_train)          # শিখলাম
3X_train = imputer.transform(X_train)  # প্রয়োগ করলাম
4

5# পদ্ধতি ২ (সংক্ষেপে)
6X_train = imputer.fit_transform(X_train)  # শিখলাম + প্রয়োগ করলাম – একলাইনে
```

টেস্ট ডেটার জন্য: শুধু transform() – কখনো fit() বা fit_transform() নয়।

KDE plot এর প্রসঙ্গ

ভিডিওর শুরুতে Mean ও Median ইম্পিউটেশনের KDE plot দেখা হয়েছিল। সিম্পল ইম্পিউটার ব্যবহার করে ইম্পিউট করার পর সেই plot-এর মতো ডিস্ট্রিবিউশন তৈরি হয়।

মনে রাখবেন:

Mean ইম্পিউটেশন → ডিস্ট্রিবিউশনের ভ্যারিয়েশন কমায় (standard deviation কমে)
Median ইম্পিউটেশন → তুলনামূলক ভালো, বিশেষ করে আউটলায়ার থাকলে

গুরুত্বপূর্ণ সতর্কতা

1. ravel() কেন দরকার?

```
# transform() 2D আউটপুট দেয়
result = imputer.transform(X_train[['Age']])  # shape: (712, 1)

# .ravel() 1D বানায়
result_1d = result.ravel()  # shape: (712,)

# তারপর কলামে অ্যাসাইন
X_train['Age'] = result_1d
```

যদি ravel() না দেন:

```
X_train['Age'] = imputer.transform(X_train[['Age']])  # error! 2D array 1D কলামে যায় না
```

2. ট্রেন ও টেস্টের জন্য আলাদা ইম্পিউটার কেন নয়?

```
# ❌ ভুল পদ্ধতি
imputer_train = SimpleImputer(strategy='mean')
imputer_test = SimpleImputer(strategy='mean')
imputer_train.fit(X_train)
imputer_test.fit(X_test)    # টেস্ট থেকে শিখলাম – লিকেজ!

# ✅ সঠিক পদ্ধতি
imputer = SimpleImputer(strategy='mean')
imputer.fit(X_train)        # শুধু ট্রেন থেকে শিখলাম
X_train = imputer.transform(X_train)
X_test = imputer.transform(X_test)  # টেস্টে শুধু প্রয়োগ
```

3. ইন্ডাস্ট্রিতে কেন সিম্পল ইম্পিউটার বেশি ইউজ হয়?

Pipeline-এর সাথে কাজ করে: কলাম ট্রান্সফরমার, পাইপলাইনে সরাসরি যুক্ত করা যায়
প্রোডাকশনে সহজ: ফিট করা ইম্পিউটার pickle করে সেভ করে রাখা যায়, পরে লোড করে ট্রান্সফর্ম করা যায়
রিপ্রোডিউসেবল: রেন্ডম স্টেট সাপোর্ট করে (কিছু স্ট্রাটেজির জন্য)

Categorical Value Imputation with Mode Value

ক্যাটাগরিক্যাল মিসিং ভ্যালু ইম্পিউটেশন – ভূমিকা

কেন আলাদা পদ্ধতি?

ক্যাটাগরিক্যাল ডেটার গড় (mean) বা মাঝের মান (median) থাকে না। তাই আলাদাভাবে হ্যান্ডেল করতে হয়।

দুটি প্রধান পদ্ধতি:

পদ্ধতি	কী করে	কখন ব্যবহার করবেন
মোড (Mode) দিয়ে ইম্পিউট	সবচেয়ে বেশি বার আসা ক্যাটাগরি বসায়	সাধারণ পরিস্থিতিতে
"Missing" স্ট্রিং দিয়ে ইম্পিউট	নতুন ক্যাটাগরি "Missing" বসায়	মিসিং থাকাটাই তথ্য হতে পারে

মোড (Mode)

মোড কী?

একটি ডেটাসেটে সবচেয়ে বেশি সংখ্যক বার আসা মান।

উদাহরণ:

```
1Embarked কলামের মান:
2S (Southampton) → 600 বার
3C (Cherbourg)   → 200 বার
4Q (Queenstown)  → 89 বার
5
6এখানে মোড = 'S' (সবচেয়ে বেশি বার এসেছে)
```

মোড দিয়ে ইম্পিউটেশন মানে:

যেখানে Embarked-এর মান নেই (NaN), সেখানে 'S' বসিয়ে দেওয়া।

Missing Indicator – কী এবং কখন ব্যবহার করবেন?

Missing Indicator কী?

একটি অতিরিক্ত কলাম তৈরি করে, যেখানে:

- যেখানে আগে মিসিং ছিল → True বা 1
- যেখানে মিসিং ছিল না → False বা 0

উদাহরণ:

আগের Embarked	ইম্পিউটেশন Embarked	Missing Indicator
S	S	0 (মিসিং ছিল না)
NaN	S	1 (মিসিং ছিল, ইম্পিউট করা হয়েছে)
C	C	0
NaN	S	1
Q	Q	0

কখন **Missing Indicator** ব্যবহার করবেন**?**

পরিস্থিতি	করবেন
খুব কম কলামে মিসিং আছে (২-৩টি)	Indicator ব্যবহার করুন → মডেল ভালো পারফর্ম করতে পারে
অনেক কলামে মিসিং আছে (১০০+টি)	Indicator ব্যবহার করবেন না → ডাইমেনশনালিটি অনেক বেড়ে যায়
মিসিং থাকাটাই তথ্যপূর্ণ	Indicator ব্যবহার করুন

কেন ডাইমেনশনালিটি বাড়ে**?**

১০০টি কলামে যদি প্রতিটির জন্য Indicator বানান, তাহলে ১০০টি বাড়তি কলাম তৈরি হবে। মডেলের জন্য অতিরিক্ত কলাম শেখা কঠিন হয়ে যায় (Curse of Dimensionality)।

টাইটানিকের Embarked-এ মাত্র ২টি মিসিং। এই ক্ষেত্রে Indicator খুব একটা কাজ করবে না (শুধু ২টি মানের জন্য আলাদা কলাম তৈরি করা বাড়তি)।

টাইটানিকের ক্যাটাগরিক্যাল কলাম

কলাম	টাইপ	মিসিং সংখ্যা	পদ্ধতি
Embarked	ক্যাটাগরিক্যাল (S/C/Q)	২টি	মোড দিয়ে ইম্পিউট
Cabin	ক্যাটাগরিক্যাল	৬৮৭টি (৭৭%)	আগেই ডিলিট করা হয়েছে

ধাপ ১: সিম্পল ইম্পিউটার ইম্পোর্ট ও কনফিগার

```
from sklearn.impute import SimpleImputer

embarked_imputer = SimpleImputer(
    missing_values=np.nan,
```

```
    strategy='most_frequent'
)
```

প্যারামিটার ব্যাখ্যা:

প্যারামিটার	মান	ব্যাখ্যা
missing_values	np.nan	কী মান মিসিং হিসেবে চিহ্নিত হবে
strategy	'most_frequent'	সবচেয়ে বেশি বার আসা মান (মোড) দিয়ে পূরণ করো

strategy-এর অন্যান্য অপশন:

strategy	ব্যবহার
'most_frequent'	ক্যাটাগরিক্যাল ডেটার জন্য (মোড)
'constant'	নির্দিষ্ট মান বসাতে চাইলে (যেমন 'Missing')

ধাপ ২: ফিট (Fit) – ট্রেনিং ডেটা থেকে মোড শেখা

```
embarked_imputer.fit(X_train[['Embarked']])
```

এখানে কী হলো?

```
1. X_train['Embarked'] কলামে যাও
2. বিদ্যমান মানগুলো দেখ (S, C, Q)
3. কোনটি সবচেয়ে বেশি বার এসেছে? → এটি মোড
4. এই মোড মানটি মেমরিতে সেভ কর
```

মনে রাখবেন: ফিট করা মানে শেখা। ট্রেনিং ডেটা থেকেই মোড বের করতে হবে, টেস্ট থেকে নয়।

2D ডেটা দেওয়া জরুরি:

```
# ✅ সঠিক (2D)
embarked_imputer.fit(X_train[['Embarked']])

# ❌ ভুল (1D) – error দিবে
embarked_imputer.fit(X_train['Embarked'])
```

ধাপ ৩: ট্রান্সফর্ম – ট্রেনিং ডেটা ইম্পিউট করা

```
X_train['Embarked'] =
embarked_imputer.transform(X_train[['Embarked']]).ravel()
```

এই লাইনের ব্যাখ্যা:

অংশ	কী করে
X_train[['Embarked']]	2D আকারে Embarked কলাম নেওয়া
embarked_imputer.transform(...)	মোড মান দিয়ে মিসিং পূরণ করা
.ravel()	2D আউটপুটকে 1D তে রূপান্তর
X_train['Embarked'] = ...	পুরোনো কলামের জায়গায় ইম্পিউটেড মান বসানো

কেন **ravel()** লাগছে?

```
# transform() 2D আউটপুট দেয়
result = imputer.transform(X_train[['Embarked']])
# result.shape = (712, 1)  ← 2D

# .ravel() 1D বানায়
result_1d = result.ravel()
# result_1d.shape = (712,)  ← 1D

# 1D কলামে 1D ডেটা বসানো যায়
X_train['Embarked'] = result_1d  # ✅
```

নিউমেরিক্যাল ইম্পিউটেশনে pandemics auto-convert করে দিলেও, ক্যাটাগরিক্যালের জন্য ravel() নিজে করতে হয়।

ধাপ ৪: টেস্টিং ডেটা ট্রান্সফর্ম করা

```
X_test['Embarked'] =
embarked_imputer.transform(X_test[['Embarked']]).ravel()
```

গুরুত্বপূর্ণ: টেস্ট ডেটার জন্য ফিট করছি না। ট্রেন থেকে শেখা মোড মান ব্যবহার করছি।

ধাপ ৫: ফলাফল যাচাই করা

```
print(X_train.isnull().sum())
print(X_test.isnull().sum())
```

ফলাফল: Embarked কলামে মিসিং সংখ্যা ০ (শূন্য) – উভয় ডেটাসেটেই।

পুরো কোড একসাথে

```
from sklearn.impute import SimpleImputer

# 1. ইম্পিউটার অবজেক্ট (মোড দিয়ে)
embarked_imputer = SimpleImputer(
    missing_values=np.nan,
    strategy='most_frequent'
)

# 2. ট্রেনিং ডেটা থেকে শেখা (ফিট)
embarked_imputer.fit(X_train[['Embarked']])

# 3. ট্রেনিং ডেটা ট্রান্সফর্ম
X_train['Embarked'] = embarked_imputer.transform(
    X_train[['Embarked']]
).ravel()

# 4. টেস্টিং ডেটা ট্রান্সফর্ম
X_test['Embarked'] = embarked_imputer.transform(
    X_test[['Embarked']]
).ravel()

# 5. যাচাই
print(X_train['Embarked'].isnull().sum())  # 0
print(X_test['Embarked'].isnull().sum())   # 0
```

গুরুত্বপূর্ণ সতর্কতা

1. ravel() ভুলে গেলে কী হবে?

```
# ভুল
X_train['Embarked'] = imputer.transform(X_train[['Embarked']])
# ValueError: setting an array element with a sequence.
```

সমাধান: সবসময় .ravel() ব্যবহার করুন।

2. মোড একাধিক থাকলে কী হয়?

যদি দুইটি ক্যাটাগরির ফ্রিকোয়েন্সি সমান হয় (টাই), Scikit-learn প্রথমটি নেয়।

এড়ানোর উপায়:

আগে নিজে চেক করে নিন:

```
mode_values = X_train['Embarked'].mode()
if len(mode_values) > 1:
    print("Warning: Multiple modes found!")
```

3. ক্যাবিন কলামের জন্য কী করা উচিত?

Cabin নিয়ে আলোচনা হয়েছে (৬৮৭টি মিসিং, ৭৭%)। এই পরিমাণ মিসিং থাকলে:

> **Mode** ইম্পিউটেশন: কাজ করবে না (মোডও মিসিং হতে পারে)
> **"Missing"** ইম্পিউটেশন: সম্ভব, কিন্তু ৭৭% ডেটা 'Missing' হলে তা তথ্যপূর্ণ নয়
> সেরা পদ্ধতি: কলাম ডিলিট করা

Categorical Value Imputation with Missing string and Indicator

"Missing" স্ট্রিং দিয়ে ইম্পিউটেশন – কেন এবং কখন?

কেন "Missing" স্ট্রিং ব্যবহার করবেন?

পরিস্থিতি	পদ্ধতি	কারণ
মিসিং ভ্যালুতে কোনো প্যাটার্ন নেই	মোড (Mode) দিয়ে ইম্পিউট	সবচেয়ে কমন মান বসানো নিরাপদ
মিসিং থাকাটাই একটি ইনফরমেশন হতে পারে	"Missing" স্ট্রিং দিয়ে ইম্পিউট	মডেল শিখতে পারে কেন ডেটা মিসিং আছে
মিসিং ভ্যালুর প্যাটার্ন বুঝতে পারছেন না	"Missing" স্ট্রিং দিয়ে ইম্পিউট	নিরাপদ পছন্দ, নতুন ক্যাটাগরি তৈরি করে
খুব বেশি মিসিং (৫০%+)	কলাম ডিলিট	Cabin এর ক্ষেত্রে ৭৭% মিসিং – ডিলিট ভালো (শেখার জন্য এখানে রেখেছেন)

NaN vs "Missing" – পার্থক্য কী?

NaN (ফাঁকা)	"Missing" (স্ট্রিং)
মডেল বুঝতে পারে না → error দিতে পারে	মডেল একটি ক্যাটাগরি হিসেবে বুঝতে পারে
শেখার কোনো সুযোগ নেই	মডেল শিখতে পারে "Missing" মানে কী
ইম্পিউট না করলে মডেল রান করে না	মডেল রান করবে, কারণ সব মান ক্যাটাগরিতে রূপান্তরিত

মূলকথা: NaN থাকলে মডেল কাজ করে না। "Missing" বসালে মডেল এটাকে একটি সাধারণ ক্যাটাগরি হিসেবে দেখে।

টাইটানিকের Cabin কলাম – কেন এই উদাহরণ?

কলাম	টাইপ	মিসিং সংখ্যা	মিসিং %
Cabin	ক্যাটাগরিক্যাল	X_train-এ 553টি, X_test-এ 134টি	প্রায় ৭৭%

Cabin রাখার কারণ:
শেখার জন্য। ইন্ডাস্ট্রিতে ৭৭% মিসিং থাকলে কলাম ডিলিট করাই ভালো। কিন্তু এখানে "Missing" স্ট্রিং ও ইন্ডিকেটর – এই দুটি কনসেপ্ট বোঝানোর জন্য Cabin কলাম ব্যবহার করা হয়েছে।

ধাপ ১: ডিস্ট্রিবিউশন বোঝা (কেন মোড কাজ করবে না?)

```
1import seaborn as sns
```

```
2sns.countplot(data=X_train, x='Cabin')
```

ফলাফল পর্যবেক্ষণ:

Cabin কলামে অনেক ভিন্ন ভিন্ন ক্যাটাগরি আছে (A, B, C, D, E, F, G, T ইত্যাদি)। কোনো ক্যাটাগরির ফ্রিকোয়েন্সি স্পষ্টভাবে বেশি না।

এই কারণে মোড **(most_frequent)** সঠিক হবে না:

> মোড = সবচেয়ে বেশি বার আসা ক্যাটাগরি
> কিন্তু এখানে সব ক্যাটাগরি প্রায় সমান
> কোনো একটি ক্যাটাগরিতে বিশেষ আধিপত্য নেই

সিদ্ধান্ত: মোড না দিয়ে "Missing" স্ট্রিং দিয়ে ইম্পিউট করবো।

ধাপ ২: সিম্পল ইম্পিউটার – কনস্ট্যান্ট স্ট্র্যাটেজি ও ইন্ডিকেটর

```
from sklearn.impute import SimpleImputer

cabin_imputer = SimpleImputer(
    missing_values=np.nan,
    strategy='constant',
    fill_value='Missing',
    add_indicator=True
)
```

প্যারামিটার ব্যাখ্যা:

প্যারামিটার	মান	ব্যাখ্যা
missing_values	np.nan	NaN-কে মিসিং হিসেবে চিহ্নিত করো
strategy	'constant'	নির্দিষ্ট একটি মান দিয়ে পূরণ করো
fill_value	'Missing'	যে মানটি বসাবে (এখানে স্ট্রিং 'Missing')
add_indicator	True	মিসিং ইন্ডিকেটর কলাম যোগ করো

add_indicator=True কী করে?

আউটপুটে দুটি কলাম রিটার্ন করে:

কলাম	কী থাকে
১ম কলাম	ইম্পিউটেড মান (মিসিং জায়গায় 'Missing' বসে)
২য় কলাম	ইন্ডিকেটর (মিসিং ছিল কিনা) – True/False

ইন্ডিকেটর কলামের মান:

পরিস্থিতি	ইন্ডিকেটর মান
আসল মান ছিল (মিসিং ছিল না)	False বা 0
আসল মান ছিল NaN (মিসিং ছিল)	True বা 1

ধাপ ৩: ফিট (Fit) – ট্রেনিং ডেটা থেকে শেখা

```
cabin_imputer.fit(X_train[['Cabin']])
```

এখানে কী শেখে?

strategy='constant' ও fill_value='Missing' দেওয়া থাকায়, ফিট খুব বেশি কিছু শেখে না – শুধু মনে রাখে 'Missing' মানটি বসাতে হবে।

নোট: add_indicator=True থাকলে, ফিট করার সময় মিসিং প্যাটার্নও শেখে (কোন কোন ইনডেক্সে মিসিং ছিল)।

ধাপ ৪: ট্রান্সফর্ম – ট্রেনিং ও টেস্টিং ডেটা ইম্পিউট করা

ট্রেনিং ডেটার জন্য:

```
X_train[['Cabin', 'cabin_missing_indicator']] =
cabin_imputer.transform(X_train[['Cabin']])
```

টেস্টিং ডেটার জন্য:

```
X_test[['Cabin', 'cabin_missing_indicator']] =
cabin_imputer.transform(X_test[['Cabin']])
```

এই লাইনের ব্যাখ্যা:

অংশ	কী করে
cabin_imputer.transform(...)	2D array আউটপুট দেয় (দুইটি কলাম)
[['Cabin', 'cabin_missing_indicator']]	দুইটি কলামের নাম নির্ধারণ
X_train[...] = ...	দুইটি নতুন কলাম X_train-এ যোগ করে

আগের মত ravel() কেন লাগছে না?

কারণ: এখন আমরা একটি কলাম ইম্পিউট করছি না – আমরা দুইটি কলাম পাচ্ছি:

- ইম্পিউটেড Cabin (মান)
- Missing Indicator (True/False)

ravel() শুধু 2D কে 1D বানায় – কিন্তু এখানে 2D আউটপুট (দুই কলাম) আমরা সরাসরি দুই কলামে বসাচ্ছি। তাই ravel() দরকার নেই।

ধাপ ৫: ফলাফল দেখা ও যাচাই

X_train দেখা:

```
1X_train
```

ফলাফল:

- Cabin কলামে আগে যেখানে NaN ছিল, এখন 'Missing' বসেছে
- নতুন cabin_missing_indicator কলাম যোগ হয়েছে
- True = আগে NaN ছিল
- False = আগে মান ছিল

X_test দেখা:

```
1X_test
```

একই পরিবর্তন – 'Missing' ও ইন্ডিকেটর কলাম যোগ হয়েছে।

মিসিং চেক করা:

```
1print(X_train.isnull().sum())
```

```
2print(X_test.isnull().sum())
```

ফলাফল: Cabin ও cabin_missing_indicator – উভয় কলামে ০ মিসিং।

পুরো কোড একসাথে

```
from sklearn.impute import SimpleImputer
import seaborn as sns

# 1. ডিস্ট্রিবিউশন দেখা (কেন মোড কাজ করবে না বোঝার জন্য)
sns.countplot(data=X_train, x='Cabin')

# 2. ইম্পিউটার তৈরি
cabin_imputer = SimpleImputer(
    missing_values=np.nan,
     strategy='constant',
     fill_value='Missing',
     add_indicator=True
)

# 3. ট্রেনিং ডেটা ফিট
cabin_imputer.fit(X_train[['Cabin']])

# 4. ট্রেনিং ডেটা ট্রান্সফর্ম
X_train[['Cabin', 'cabin_missing_indicator']] =
cabin_imputer.transform(X_train[['Cabin']])

# 5. টেস্টিং ডেটা ট্রান্সফর্ম
X_test[['Cabin', 'cabin_missing_indicator']] =
cabin_imputer.transform(X_test[['Cabin']])

# 6. যাচাই
print(X_train.isnull().sum())
print(X_test.isnull().sum())
```

ভিজুয়াল ধারণা – ইন্ডিকেটর কীভাবে কাজ করে

BEFORE (Cabin কলাম):

```
Index    Cabin
```

```
20          B5
31          NaN     ← মিসিং
42          C23
53          NaN     ← মিসিং
64          E8
```

AFTER (Transform করে দুই কলাম**):**

Index	Cabin (ইম্পিউটেড)	cabin_missing_indicator
0	B5	False (মিসিং ছিল না)
1	Missing	True (মিসিং ছিল)
2	C23	False
3	Missing	True (মিসিং ছিল)
4	E8	False

Constant Strategy vs Most Frequent – কখন কোনটা**?**

পরিস্থিতি	সঠিক পদ্ধতি	উদাহরণ
একটি ক্যাটাগরি স্পষ্টভাবে বেশি	strategy='most_frequent'	Embarked (S বেশি)
সব ক্যাটাগরি প্রায় সমান	strategy='constant', fill_value='Missing'	Cabin (সব কাছাকাছি)
মিসিং থাকাটা ইনফরমেশন	strategy='constant', fill_value='Missing'	সাধারণভাবে ভালো পছন্দ
ডোমেইন নলেজ আছে (যেমন: 'None')	strategy='constant', fill_value='None'	নির্দিষ্ট মান জানা থাকলে

গুরুত্বপূর্ণ সতর্কতা

1. ইন্ডিকেটর কখন ব্যবহার করবেন?

ইন্ডিকেটর ব্যবহার করবেন	ইন্ডিকেটর ব্যবহার করবেন না
২-৩টি কলামে মিসিং থাকলে	১০০+ কলামে মিসিং থাকলে
মিসিং থাকাটা গুরুত্বপূর্ণ মনে হলে	ডাইমেনশনালিটি বাড়াতে না চাইলে
এক্সপেরিমেন্ট করছেন (টেস্ট করে দেখতে চান)	মডেল ইতিমধ্যে ভালো কাজ করলে

2. add_indicator=False (ডিফল্ট) থাকলে কী হয়?

```
imputer = SimpleImputer(strategy='constant', fill_value='Missing')
# add_indicator=False ডিফল্ট

X_train['Cabin'] = imputer.fit_transform(X_train[['Cabin']]).ravel()
# শুধু ইম্পিউটেড মান পাবেন, ইন্ডিকেটর পাবেন না
```

3. নিউমেরিক্যাল কলামেও Constant Strategy ব্যবহার করা যায়

```
# নিউমেরিক্যালে -1 দিয়ে ইম্পিউট
imputer = SimpleImputer(strategy='constant', fill_value=-1)
X_train['Age'] = imputer.fit_transform(X_train[['Age']]).ravel()
```

4. Cabin-এর ক্ষেত্রে ইন্ডাস্ট্রি প্র্যাকটিস কী?

Cabin-এ ৭৭% মিসিং – ইন্ডাস্ট্রিতে সাধারণত:

কলাম ডিলিট – এটাই বেস্ট প্র্যাকটিস
বা 'Missing' দিয়ে ইম্পিউট করে দেখা – যদি এক্সপেরিমেন্ট করতে চান

শেখানোর জন্য Cabin রাখা হয়েছে, প্রোডাকশনে ডিলিট করাই ভালো।